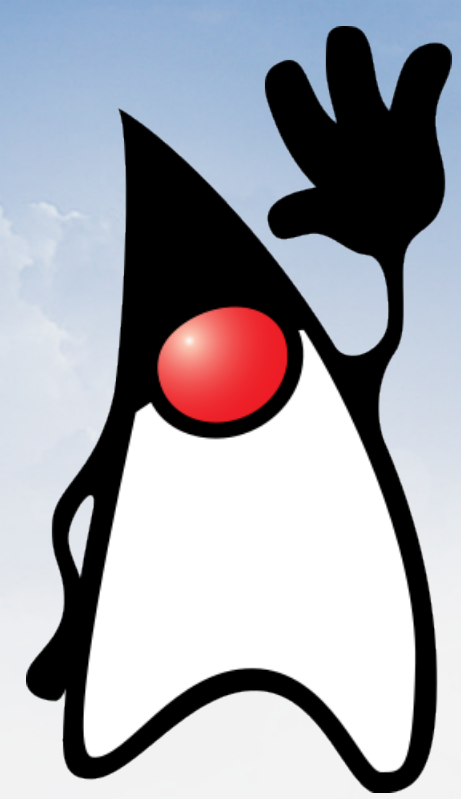




After OOP, confessions

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

primitive, non primitive

- primitive variables are still primitive
- non primitive variable means
 - They are objects!
- That's why we need to use equals to check equality between them

Strings

- Strings are objects that hold the char array and its length
- It has even toString() method
- Although the size is constant, we get this by method *.length()*
- in fact we should define it as `String name=new String("test");`

```
public final class String{  
    int length;  
    char[] arr;  
  
    String(..)  
  
}
```

arrays

We define arrays as if they are objects.
`int[] arr=new int[num];`

similar with
`String s=new String("");`

length is a public **final** variable in array object. not a method.

That's why we use it as length
No paranthesis!

```
public class HelloWorld
{
    public static void main(String[] args)
    {
        int[] someIntegers = new int[42];
        int length=someIntegers.length;
        System.out.println(someIntegers instanceof Object); //True
    }
}
```

ArrayList

- ArrayList is an object, holds array with variable length
- Extends the array as needed.
- but it is taking some generic to be put inside.
- `ArrayList<String> arr=new ArrayList<String>();`
- ArrayList extends the size but does not shrink.

Random, Scanner are objects

- Random random =new Random();
 - random.nextInt() is a method
- Random somethingRandom=new Random();
- Scanner anotherScanner=new Scanner();
- Math is an object but all methods are static
 - Probably abstract or private constructor
 - We can't create instance of Math class ~~Math m=new Math();~~
 - That's why we don't create it. we just call it like Math.sin()
 - Pi is a **final** instance variable in Math class **Math.PI**

main method

- main method is not a method of any object,
- It should be runnable/callable by anything
- So it is static!

pass by what?

```
public class myobj {  
    public int val;  
  
    public myobj(){  
        val=4;  
    }  
}
```

```
before 4  
after 5  
false  
false
```

Copy of reference is passed. we can modify but we can not null it!

```
public class myobjTester {  
  
    public static void main(String[] args) {  
        myobj c=new myobj();  
        System.out.println("before " + c.val);  
        changeVal(c);  
        System.out.println("after " + c.val);  
        System.out.println("before " + c==null);  
        killer(c);  
        System.out.println("after " + c == null);  
    }  
  
    public static void changeVal(myobj x2){  
        x2.val=5;  
    }  
  
    public static void killer(myobj x1){  
        x1=null;  
    }  
}
```

Our classes

- They were all object
- We werent using object features, just writing static methods
- We can call by the name of the class

```
public class myobjTester {  
  
    public static void main(String[] args) {  
        myobj c=new myobj();  
        System.out.println("before " + c.val);  
        myobjTester.changeVal(c);  
        System.out.println("after " + c.val);  
        System.out.println("before " + c==null);  
        myobjTester.killer(c);  
        System.out.println("after " + c == null);  
    }  
  
    public static void changeVal(myobj x2){  
        x2.val=5;  
    }  
  
    public static void killer(myobj x1){  
        x1=null;  
    }  
  
}
```

imports

- `import java.util.Scanner;`
 - there is a java package, holds classes(?) and packages
 - inside there is util package, holds many classes
 - inside there is a Scanner class
- `import java.lang.*;`
 - is useless. It is imported by default
 - it has String inside, System.out inside

static import (NO!)

```
import static java.lang.Math.*;
import static java.lang.System.out;

public class MathExample {
    public static void main(String[] args) {
        double result = sqrt(16);
        // Direct use of sqrt(), without Math prefix
        System.out.println(result);
        out.println("Hello, world!");
    }
}
```

DO NOT USE Static import. In this course, you will lose points if you do.

The reader will try to guess where the sqrt() is important. The code will not be readable!

System.out.println()

System.out.println(someObject); is translated as

System.out.println(someObject.toString());

Compiler does that automatically.

thats why you can print a StringBuilder without toString

System.out.println(new StringBuilder("Java"));